Day 20 — File Formats and Output : Scenario: Store daily sales partitioned by year/month/day.

COMMANDS :

```
mkdir -p ~/day20-file-formats/src/main/scala
cd ~/day20-file-formats

nano build.sbt

ThisBuild / version := "0.1.0"
ThisBuild / scalaVersion := "2.12.18"
lazy val root = (project in file("."))
  .settings(
    name := "Day20-File-Formats",
    libraryDependencies ++= Seq(
      "org.apache.spark" %% "spark-core" % "3.5.3",
      "org.apache.spark" %% "spark-sql" % "3.5.3"
    ),
    Compile / run / fork := true,
    Compile / run / javaOptions ++= Seq(
      "--add-exports=java.base/sun.nio.ch=ALL-UNNAMED"
    )
  )

nano src/main/scala/FileFormats.scala

import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import java.io.File
object FileFormats {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder()
      .appName("Day20 File Formats")
      .master("local[*]")
      .getOrCreate()
    spark.sparkContext.setLogLevel("WARN")
    import spark.implicits._
    val sales = Seq(
      ("S001", "P001", 101, 500.0, "2026-10-01"),
      ("S002", "P002", 102, 750.0, "2026-10-01"),
      ("S003", "P003", 101, 1200.0, "2026-10-01"),
      ("S004", "P004", 103, 900.0, "2026-10-02"),
      ("S005", "P005", 102, 1500.0, "2026-10-02"),
```

```
  ("S006", "P006", 104, 2000.0, "2026-10-02"),
  ("S007", "P007", 101, 650.0, "2026-10-03"),
  ("S008", "P008", 103, 1100.0, "2026-10-03"),
  ("S009", "P009", 102, 850.0, "2026-10-03"),
  ("S010", "P010", 104, 1750.0, "2026-10-04")
)
val salesDF = sales.toDF(
  "sale_id",
  "product_id",
  "branch_id",
  "amount",
  "sale_date"
)
val dailySalesDF = salesDF
  .withColumn(
    "sale_date",
    to_date(col("sale_date"))
  )
  .withColumn(
    "year",
    year(col("sale_date"))
  )
  .withColumn(
    "month",
    month(col("sale_date"))
  )
  .withColumn(
    "day",
    dayofmonth(col("sale_date"))
  )
println("\n========== SALES DATA ==========")
dailySalesDF.show(false)
println("\n========== WRITING CSV ==========")
dailySalesDF
  .drop("year", "month", "day")
  .write
  .mode("overwrite")
  .option("header", "true")
  .csv("output/csv")
println("\n========== READING CSV ==========")
val csvDF = spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("output/csv")
```

```
  csvDF.show(false)
  println("\n========== WRITING JSON ==========")
  dailySalesDF
    .drop("year", "month", "day")
    .write
    .mode("overwrite")
    .json("output/json")
  println("\n========== READING JSON ==========")
  val jsonDF = spark.read
    .json("output/json")
  jsonDF.show(false)
  println("\n========== WRITING PARQUET ==========")
  dailySalesDF
    .drop("year", "month", "day")
    .write
    .mode("overwrite")
    .parquet("output/parquet")
  println("\n========== READING PARQUET ==========")
  val parquetDF = spark.read
    .parquet("output/parquet")
  parquetDF.show(false)
  println("\n========== REPARTITION ==========")
  val repartitionedDF = dailySalesDF
    .repartition(4)
  println(
    "Number of partitions after repartition = " +
    repartitionedDF.rdd.getNumPartitions
  )
  println("\n========== WRITING PARTITIONED PARQUET ==========")
  repartitionedDF
    .write
    .mode("overwrite")
    .partitionBy("year", "month", "day")
    .parquet("output/daily_sales")
  println("\n========== READING PARTITIONED PARQUET ==========")
  val partitionedDF = spark.read
    .parquet("output/daily_sales")
  partitionedDF.show(false)
  println(
    "\nNumber of partitions in DataFrame = " +
    partitionedDF.rdd.getNumPartitions
  )
  spark.stop()
}
```

}

sbt clean
sbt compile
sbt run

OUTPUTS :

```
========== SALES DATA ==========
+-------+----------+---------+------+----------+----+-----+---+
|sale_id|product_id|branch_id|amount|sale_date |year|month|day|
+-------+----------+---------+------+----------+----+-----+---+
|S001   |P001      |101      |500.0 |2026-10-01|2026|10   |1  |
|S002   |P002      |102      |750.0 |2026-10-01|2026|10   |1  |
|S003   |P003      |101      |1200.0|2026-10-01|2026|10   |1  |
|S004   |P004      |103      |900.0 |2026-10-02|2026|10   |2  |
|S005   |P005      |102      |1500.0|2026-10-02|2026|10   |2  |
|S006   |P006      |104      |2000.0|2026-10-02|2026|10   |2  |
|S007   |P007      |101      |650.0 |2026-10-03|2026|10   |3  |
|S008   |P008      |103      |1100.0|2026-10-03|2026|10   |3  |
|S009   |P009      |102      |850.0 |2026-10-03|2026|10   |3  |
|S010   |P010      |104      |1750.0|2026-10-04|2026|10   |4  |
+-------+----------+---------+------+----------+----+-----+---+


========== WRITING CSV ==========

========== READING CSV ==========
+-------+----------+---------+------+----------+
|sale_id|product_id|branch_id|amount|sale_date |
+-------+----------+---------+------+----------+
|S006   |P006      |104      |2000.0|2026-10-02|
|S003   |P003      |101      |1200.0|2026-10-01|
|S010   |P010      |104      |1750.0|2026-10-04|
|S005   |P005      |102      |1500.0|2026-10-02|
|S008   |P008      |103      |1100.0|2026-10-03|
|S001   |P001      |101      |500.0 |2026-10-01|
|S007   |P007      |101      |650.0 |2026-10-03|
|S002   |P002      |102      |750.0 |2026-10-01|
|S004   |P004      |103      |900.0 |2026-10-02|
|S009   |P009      |102      |850.0 |2026-10-03|
+-------+----------+---------+------+----------+
```

```
========== WRITING JSON ==========

========== READING JSON ==========
+------+---------+----------+----------+-------+
|amount|branch_id|product_id|sale_date |sale_id|
+------+---------+----------+----------+-------+
|1500.0|102      |P005      |2026-10-02|S005   |
|1100.0|103      |P008      |2026-10-03|S008   |
|1200.0|101      |P003      |2026-10-01|S003   |
|2000.0|104      |P006      |2026-10-02|S006   |
|1750.0|104      |P010      |2026-10-04|S010   |
|750.0 |102      |P002      |2026-10-01|S002   |
|500.0 |101      |P001      |2026-10-01|S001   |
|900.0 |103      |P004      |2026-10-02|S004   |
|850.0 |102      |P009      |2026-10-03|S009   |
|650.0 |101      |P007      |2026-10-03|S007   |
+------+---------+----------+----------+-------+


========== WRITING PARQUET ==========

========== READING PARQUET ==========
+-------+----------+---------+------+----------+
|sale_id|product_id|branch_id|amount|sale_date |
+-------+----------+---------+------+----------+
|S008   |P008      |103      |1100.0|2026-10-03|
|S010   |P010      |104      |1750.0|2026-10-04|
|S003   |P003      |101      |1200.0|2026-10-01|
|S005   |P005      |102      |1500.0|2026-10-02|
|S009   |P009      |102      |850.0 |2026-10-03|
|S001   |P001      |101      |500.0 |2026-10-01|
|S007   |P007      |101      |650.0 |2026-10-03|
|S004   |P004      |103      |900.0 |2026-10-02|
|S006   |P006      |104      |2000.0|2026-10-02|
|S002   |P002      |102      |750.0 |2026-10-01|
+-------+----------+---------+------+----------+
```

```
========== REPARTITION ==========
Number of partitions after repartition = 4

========== WRITING PARTITIONED PARQUET ==========

========== READING PARTITIONED PARQUET ==========
+-------+----------+---------+------+----------+----+-----+---+
|sale_id|product_id|branch_id|amount|sale_date |year|month|day|
+-------+----------+---------+------+----------+----+-----+---+
|S001   |P001      |101      |500.0 |2026-10-01|2026|10   |1  |
|S002   |P002      |102      |750.0 |2026-10-01|2026|10   |1  |
|S004   |P004      |103      |900.0 |2026-10-02|2026|10   |2  |
|S005   |P005      |102      |1500.0|2026-10-02|2026|10   |2  |
|S003   |P003      |101      |1200.0|2026-10-01|2026|10   |1  |
|S010   |P010      |104      |1750.0|2026-10-04|2026|10   |4  |
|S008   |P008      |103      |1100.0|2026-10-03|2026|10   |3  |
|S009   |P009      |102      |850.0 |2026-10-03|2026|10   |3  |
|S007   |P007      |101      |650.0 |2026-10-03|2026|10   |3  |
|S006   |P006      |104      |2000.0|2026-10-02|2026|10   |2  |
+-------+----------+---------+------+----------+----+-----+---+
```